

به نام خدا

هدف از سناریوی عملی ارزیابی توانایی شرکت‌کنندگان در طراحی، پیاده‌سازی، استقرار، پایش، عیب‌یابی و نگهداری یک سامانه Cloud Native مطابق استانداردهای DevOps است.

1. قوانین کلی سناریوی عملی مسابقه.....2
2. سناریو هفته نخست.....3
 - 2.1 آماده‌سازی زیرساخت.....3
 - مدیریت کد و همکاری تیمی.....3
 - مدیریت Secrets.....3
 - توسعه Ansible Collection.....4
 - Provisioning مانیتورینگ.....4
 - Alerting Rule.....5
 - Webhook Notification Service.....5
 - 2.2 استقرار نخستین نسخه نرم‌افزار.....6
 - 2.2.1 Shift-Left Quality کنترل کیفیت کد.....6
 - 2.2.2 Pipeline Shift-Left Security امنیت Pipeline.....7
4. راهنمای آماده سازی کانال بله جهت هشداردهی.....9

1. قوانین کلی سناریوی عملی مسابقه

- همه شرکت کنندگان موظف اند در سیستم احراز هویت مرکزی به آدرس sso.amertka.com حساب کاربری با فرمت **firstname.lastname** ایجاد کنند.
- تمامی تغییرات باید از طریق GitLab و CI/CD انجام شوند.
- همه Configuration ها ، Source Code ها باید داخل GitLab نگهداری شود.
- تمامی Repository ها باید دارای README باشند.
- شرکت کنندگان مجاز هستند برای یافتن بهترین راهکارها از مستندات رسمی و منابع معتبر اینترنت استفاده کنند.
- همه شرکت کنندگان موظف اند Image ها و Package ها و Library های مورد نیاز خود را از Repository Registry برگزار کننده مسابقات تهیه کنند.

Python Package Repository	Operating System Repository
• pbr.amertka.com/pyhub	• pbr.amertka.com/debian • pbr.amertka.com/ubuntu

- تمامی نام کاربری و پسورد و دیگر داده های مورد نیاز شرکت کنندگان در سامانه **کلید** در **سازمان League-DevOps** به آدرس kelid.amertka.com توسط برگزار کننده قرار داده خواهد شد.
- به منظور اجرای **pipeline** های CI/CD میتوانید از Runner Tag با نام **ady_runner** استفاده کنید

2. سناریو هفته نخست

شرکت Ameritka در حال مهاجرت از زیرساخت سنتی به معماری Cloud-Native است. نخستین سرویس این شرکت آماده ورود به چرخه انتشار بوده و تیم DevOps مسئول طراحی، پیاده‌سازی و آماده‌سازی پلتفرمی استاندارد برای توسعه، استقرار، مانیتورینگ و بهره‌برداری از این سرویس است. هدف این مرحله، ایجاد بستری استاندارد، امن، خودکار و قابل‌تکرار برای مدیریت چرخه عمر نرم‌افزار است؛ به‌گونه‌ای که تمامی فرآیندهای زیرساختی و انتشار نرم‌افزار بر پایه Infrastructure as Code و CI/CD Automation مدیریت شوند.

2.1 آماده‌سازی زیرساخت

در این مرحله، تیم DevOps موظف است زیرساخت‌های پایه موردنیاز برای توسعه و استقرار سرویس‌ها را آماده‌سازی و استانداردسازی کند.

مدیریت کد و همکاری تیمی

سامانه Source Control Management یا SCM با آدرس git.amertka.com آماده‌سازی شده باید به همراه ساختار استاندارد پروژه‌ها، مدیریت کاربران، سطوح دسترسی، Branch Strategy و فرآیندهای همکاری تیمی راه‌اندازی و پیکربندی شود تا تمامی تغییرات زیرساخت و نرم‌افزار از طریق مخازن Git مدیریت شوند.

انتظارات:

- ایجاد Group و Project ها (گروه اصلی باید با نام تیم یکی باشد)
 - استفاده از Merge Request برای دیپلوی تمامی تغییرات
 - Branch Strategy براساس Gitflow
 - تعریف و مدیریت دسترسی اعضای تیم
 - قرار دادن Ameritka Gitlab Webhook برای هر پروژه
- نکته : اضافه کردن داوران به گروه هر تیم ها الزامی میباشد

مدیریت Secrets

به‌منظور نگهداری اطلاعات محرمانه سامانه کلید (محل نگهداری اطلاعات حساس) راه‌اندازی شده تا اطلاعاتی نظیر Credential ها، Password ها، API Token ها، Private Key ها و سایر داده‌های حساس در محل‌های نامربوط و ناامن نگهداری نشود

انتظارات:

- هر تیم دارای یک Organization و Collections های مورد نیاز بین هم تیمی ها با دسترسی مناسب
- نکته : اضافه کردن داوران به عنوان مالک سازمان

توسعه Ansible Collection

در راستای پیاده‌سازی رویکرد Infrastructure as Code ، یک Ansible Collection مطابق استانداردهای Ansible Galaxy توسعه داده شود که قابلیت آماده‌سازی، پیکربندی و مدیریت اجزای زیرساخت را فراهم کند. این Collection حداقل باید شامل Role های زیر باشد:

- Linux Management

- ایجاد و حذف کاربران سیستم‌عامل
- پیکربندی Repository های Debian
- نصب Package های پایه شامل:

- vim

- curl

- ca-certificates

- Kubernetes Management

- استقرار یک سرویس وب Nginx روی Kubernetes با استفاده از Ansible

انتظارات:

- انتشار collection در Galaxy آمریکا

Provisioning مانیتورینگ

تمامی تنظیمات و منابع Grafana باید به صورت Declarative و با استفاده از Opentufو مدیریت شوند تا ایجاد Dashboard ها، Data Source ها و سایر تنظیمات به صورت نسخه بندی شده و قابل تکرار انجام گیرد. همچنین پس از لاگین به سرویس گرافانا با استفاده از SSO به تیم پشتیبانی اعلام نمایید تا دسترسی به سازمان مربوطه را اعمال کند.

Address	Dashboard
https://pbr.amertka.com/rawhub/Dashboards/FlaskMonitoring.json	Application
https://pbr.amertka.com/rawhub/Dashboards/KubeNamespaceMonitoring.json	Kubernetes

انتظارات:

- پیکربندی Organization هر تیم در Grafana توسط Opentufو باید انجام شود.
- Dashboard ها با پوشه بندی Infrastructure و Application به طور جدا گانه باید باشند
 - Infrastructure
 - Application

Alerting Rule

زیرساخت مانیتورینگ شامل Prometheus، Grafana و Alertmanager از پیش آماده شده است.

هر تیم موظف است یک Repository مستقل برای مدیریت Alerting Rules ایجاد کرده و تمامی قوانین هشداردهی موردنیاز را به صورت فایل‌های Version Controlled در آن نگهداری کند. پس از تکمیل و اعتبارسنجی Ruleها، تغییرات باید از طریق Merge Request به شاخه Main ارسال شوند. فرآیند بررسی، بازبینی فنی (Review) و اعمال نهایی تغییرات توسط تیم پشتیبانی انجام خواهد شد و پس از تأیید، قوانین جدید در محیط مانیتورینگ Deploy و فعال می‌شوند. به منظور مشاهده وضعیت Alertها، بررسی رخدادها و فعال و مدیریت Notificationها، می‌توانید از طریق آدرس زیر به سرویس Alertmanager دسترسی داشته باشید:

جهت دسترسی Pipeline های تیم زیرساخت به Alert Rule های هر تیم . Access Token پروژه با دسترسی مناسب در اختیار تیم پشتیبانی قرار دهید.

انتظارات:

- طراحی و پیاده‌سازی Prometheus Alert Rules برای Flask
 - Flask Service Down
 - High HTTP 5xx Error Rate
 - Performance
 - Too Many Requests
 - Exception Rate
 - Database Connection Failure
- دسته‌بندی هشدارها بر اساس Severity
- پوشش رویدادهای مرتبط با زیرساخت، Kubernetes و سرویس کاربردی

Webhook Notification Service

هر تیم باید یک سرویس Webhook توسعه دهد که پس از دریافت رویداد از Alertmanager، به صورت خودکار پیام هشدار را در پیام‌رسان بله ارسال کند.

این سرویس باید قابلیت دریافت Alert های مختلف، پردازش داده‌های ورودی و تولید پیام مناسب برای تیم عملیاتی را داشته باشد.

انتظارات:

- توسعه Webhook اختصاصی توسط هر تیم
- ارسال هشدارها به پیام‌رسان بله
- استقرار سرویس بر روی Kubernetes در Namespace اختصاصی تیم
- دسترسی از طریق دامنه

2.2 استقرار نخستین نسخه نرم افزار

پس از آماده سازی زیرساخت، نخستین نسخه نرم افزار باید با استفاده از Pipeline های CI/CD تولید، اعتبارسنجی و منتشر شود. فرآیند استقرار باید بر پایه GitOps و با استفاده از Argo CD انجام گیرد تا وضعیت محیط عملیاتی همواره با وضعیت مخزن Git همگام باشد. در پایان از راه [league.amertka.com <Name Team>](https://git.amertka.com/league-devops/league-scenario) در دسترس باشد.

برای مشاهد و دریافت کد به این آدرس مراجعه کنید <https://git.amertka.com/league-devops/league-scenario>

انتظارات:

- آماده سازی Kubernetes Manifest ها
- اجرای تست های نرم افزاری
- اجرای کنترل های امنیتی
- Build و انتشار Container Image
- استقرار بر روی Kubernetes
- مدیریت Release ها از طریق GitOps و Argo CD

2.2.1 کنترل کیفیت کد Shift-Left Quality

به منظور اجرای رویکرد Shift-Left، پیش از Push هرگونه تغییر به مخزن Git، تمامی کدهای Python باید به صورت خودکار بررسی شده و تمامی خطاها و مغایرت ها پیش از ارسال کد برطرف شوند. با استفاده از ابزار Ruff حداقل کنترل های مورد نیاز عبارت اند از:

- سازگاری کامل با Python 3.11
- رعایت استانداردهای PEP 8
- حداکثر طول ۱۲۰ کاراکتر برای هر خط
- شناسایی متغیرهای تعریف نشده (Undefined Variables)
- حذف متغیرهای استفاده نشده (Unused Variables)
- حذف Export های استفاده نشده (Unused Imports)
- مرتب سازی و گروه بندی Import ها
- به روزرسانی Syntax های قدیمی و استفاده از قابلیت های Python 3.11
- ساده سازی و بهینه سازی ساختار کد (Code Simplification)
- شناسایی پارامترهای استفاده نشده در توابع
- استفاده یکنواخت از Double Quotes
- افزایش خوانایی، نگهداشت پذیری و یکپارچگی کد
- شناسایی آسیب پذیری های امنیتی و خطاهای منطقی رایج

2.2.2 کنترل‌های امنیتی Pipeline Shift-Left Security

پس از Push کد به سامانه SCM کد باید تحلیل و حداقل کنترل‌های زیر را اعمال کند: (با استفاده از ابزار Semgrep)

- شناسایی فعال بودن Debug Mode در محیط Production
 - شناسایی آسیب‌پذیری Path Traversal
 - شناسایی Cross-Site Scripting XSS
 - شناسایی Dangerous Deserialization
 - شناسایی استفاده نامن از subprocess
 - شناسایی OS Command Execution
 - شناسایی Command Injection
 - شناسایی SQL Injection
 - شناسایی Hardcoded Secrets شامل Password, API Key, Access Token, Private Key, Connection String و سایر اطلاعات محرمانه
- در صورت شناسایی هر یک از موارد فوق، Pipeline باید با وضعیت Failed خاتمه یافته و از ادامه مراحل Build, Merge و Release تا زمان رفع کامل مشکلات جلوگیری کند.

2.3 برآیند سناریوی هفته نخست

- Git Repository Structure
- Ansible Collection
- Terraform Project
- Kubernetes Manifests
- CI/CD Pipelines
- Grafana Dashboards
- Prometheus Alert Rules
- Webhook Notification Service
- GitOps Deployment Configuration
- Documentation

Team-name (GitLab Group)

└─ infrastructure (Sub Group)

└─ OpenTofu (Project)

└─ dashboards/

└─ .gitlab-ci.yml

└─ README.md

└─ ansible (Project)

└─ ansible/

└─ <name of team> (this is namespace name)

└─ <collection name> (this is collection name)

└─ docs/

└─ README.md

└─ roles/

└─ kubernetes/

└─ linux/

└─ .gitlab-ci.yml

└─ README.md

└─ prometheus-alert-rules (Project)

└─ rules/

└─ application-alert.yml

└─ .gitlab-ci.yml

└─ README.md

└─ webhook-send-alert (Project)

└─ main.py

└─ .gitlab-ci.yml

└─ requirements.txt

└─ Dockerfile

└─ README.md

└─ application (Project)

└─ source/

└─ pipelines/

└─ Dockerfile

└─ requirements.txt

└─ .gitlab-ci.yml

└─ README.md

4. راهنمای آماده سازی کانال بله جهت هشداردهی

1. ساخت کانال در پیام رسان بله با فرمت نام **Name Team_Alert**
2. عضو کردن داروان و هم تیمی ها و فرستادن نخستین پیام (**پاینده ایران**)
3. ساخت بازو بله (نکته : شما باید با دادن برخی اطلاعات به سطح کاربری را افزایش دهید)
 - a. در اپلیکیشن بله، حساب **@botfather** را جستجو و گفتگو را /start را بنویس.
 - b. نام بازو را وارد کنید با فرمت **leauge_devops_Name Team_alert**
 - c. نام کاربری را وارد کنید **league_devops_Name Team_bot**
 - d. پس از اینکه گام ها را درست پیش رفته باشید ، یک سری داده ها بهتون میده که در جایی مطمئن نگهداری کنید .
 - e. سپس روی دکمه **عضویت بازو در گروه را بزنید** و سپس دکمه **فعال سازی** را بزنید.
4. سپس به کانال ساخته شده (که در گام ۱ گفته شد) رفته و بازوی ساخته شده به عضو کنید و دسترسی ادمین بدهید. (نام بازو در گام 3 < b گفته شده که در بخش افزودن عضو جدید با جستجو میتونید نام بازو را پیدا کنید)
5. آماده سازی کد پایتون (چت آیدی توسط تیم پشتیبانی لیگ در اختیار شما قرار خواهد گرفت)

نمونه کد

```
import requests
TOKEN = ""
url = f"https://tapi.bale.ai/bot{TOKEN}/sendMessage"
payload = {
    "chat_id": "CHAT_ID", # or channel ID bale .
    "text": "درد این نخستین پیام تیم با بازو بله است"
}
response = requests.post(url, json=payload)
print(response.status_code)
print(response.text)
```

نکته : Template یا الگو Alertmanager برای فرستادن هشدارها به کانال بله بر این اساس میباشد

```
{{ define "bale.message" }}
```

Alert Notification

Alert: {{ .CommonLabels.alertname }}

Team: {{ .CommonLabels.team }}

Severity: {{ .CommonLabels.severity }}

Status: {{ .Status }}

{{ range .Alerts }}

Description:

{{ .Annotations.description }}

Instance:

{{ .Labels.instance }}

Started:

{{ .StartsAt }}

{{ end }}

{{ end }}